SENSEI PLATFORM

Capabilities, Limitations & Multi-Model Scaling Plan

Document Type: Internal Technical Guide
Audience: Engineering, Architecture Review, Team Leads
Classification: Confidential – Internal Use Only
Version: 1.0

PURPOSE OF THIS DOCUMENT

This document provides a detailed but practical view of what the Sensei platform can do today, what constraints currently exist, what constraints are likely as traffic and model count grow, and the recommended future multi-model architecture. It is written for engineering planning, architecture review, and team-lead level decision making.

1 Current System Snapshot

1.1 Product Scope

Sensei currently supports two distinct chat experiences:

- Assistant Mode — Runtime-oriented assistant flow (calculator route)
- Analyzer Mode — Telemetry and log analysis flow (analyzer route)

1.2 Technology Stack

Layer	Technology / Service
Frontend	TypeScript + React 18 + Vite + Amplify Authenticator
Backend	AWS Lambda (Python) behind API Gateway WebSocket routes
Model Access	Amazon Bedrock – Claude Sonnet 4.6 inference profile
Data Sources	CloudWatch Logs Insights, Evaluator logs, optional X-Ray details
Infrastructure	AWS SAM / CloudFormation

1.3 AWS Resource Reference

The following identifiers correspond to the current deployment environment:

Resource	Value
AgentCore Runtime ID	my_agent1-EuvQcG3t0u
AgentCore Runtime ARN	arn:aws:bedrock-agentcore:us-east-1:636052469006:runtime/...
Runtime Log Group	/aws/bedrock-agentcore/runtimes/my_agent1-EuvQcG3t0u-DEFAULT
Runtime Stream Filter (Analyzer)	agent-traces
Evaluator Log Group	/aws/bedrock-agentcore/evaluations/results/evaluation_quick_start_...
Evaluator Family Base Name	evaluation_quick_start_1773400924069
Analyzer Audit Log Group	/aws/sensei/analyzer-websocket
Analyzer Audit Stream	analyser-websocket
Data Bucket	my-agent1-price-catalog-636052469006-us-east-1

2 Current Features & Capabilities

2.1 Intent-Aware Analysis

The Analyzer classifies incoming prompts into intent categories and plans the retrieval strategy accordingly. Supported intent categories:

- summary
- listing
- deep_dive
- comparison
- anomaly
- other

Benefits:

  - Faster response for lightweight intents
  - Richer context for deep-dive intents
  - Better UX for natural-language, non-technical prompts

2.2 Adaptive Data Collection

The Analyzer dynamically decides whether to fetch each of the following sources based on intent:

- Runtime events
- Evaluator events
- X-Ray traces and subsegments

This avoids heavy fetches when not needed, improves latency for listing-style requests, and improves quality for trace diagnostics when deep details are requested.

2.3 Context Shaping Before Model Call

The system slices context to intent-appropriate payloads — rows vs. full sessions — giving control over prompt size, model cost, and the response quality vs. speed tradeoff.

2.4 WebSocket Streaming & Pagination

- Response chunking keeps payloads within transport limits
- Continuation tokens support pagination for long listings

2.5 Operational Audit Trail

The Analyzer emits stage-level audit logs at the following checkpoints:

Audit Event	Purpose
request_received	Entry point — records incoming prompt and session context
plan_inferred	Records the classified intent and planned retrieval strategy

Audit Event	Purpose
data_collection_strategy_selected	Documents which sources will be queried
context_sent_to_model	Captures the final context payload sent to Bedrock
analysis_step_before_model_invoke	Pre-invocation checkpoint for latency tracking
response_sent / timeouts / exceptions	Terminal states for every request lifecycle

These logs provide demo transparency, accelerate debugging, and enable root-cause analysis for latency or timeout incidents.

2.6 Timeout Fallback Behaviour

Timeout fallbacks ensure some requests can still return deterministic outputs when model narration times out. This meaningfully increases reliability under variable model latency.

3 Current Limitations

3.1 Request-Path Coupling to Live Queries

The Analyzer depends on live CloudWatch query cycles at request time.

> **RISK**
> Tail latency spikes, response variability, and greater timeout probability under load.

3.2 Single-Path Lambda Budget Pressure

One request path currently carries: planning, data retrieval, merge/shape, model generation, WebSocket delivery, and audit logging.

> **RISK**
> Small latency increases in each step can accumulate into end-to-end timeout.

3.3 Prompt / Context Size Sensitivity

Even with bounded token logic, quality and latency are sensitive to payload size and shape.

> **RISK**
> Inconsistent model completion time and occasional timeout for deep-dive narratives.

3.4 Partial Duplication in Utility Functions

Some WebSocket and helper logic exists in multiple Lambdas.

> **RISK**
> Maintenance drift and duplicated bug-fix effort across deployments.

3.5 Operational Complexity During Traffic Spikes

Model latency from Bedrock can vary by demand and inference profile pressure.

> **RISK**
> Periodic timeout bursts even when code is stable.

4 Future Limitations at Scale

When integrating more models and significantly higher data volume, the following become the primary scaling concerns:

4.1 Multi-Model Routing Complexity

More models means routing decisions across intent, cost, latency, quality requirements, and region/profile availability. Without a dedicated routing layer, this logic becomes brittle when consolidated in a single Lambda.

4.2 Cost Volatility

Different models and token footprints produce different cost profiles. At high volume, poor routing can rapidly increase spend, and retries or timeouts further magnify costs.

4.3 Data Volume & Query Saturation

Large historical datasets increase live query times. At scale, a pure request-time query strategy results in high p95/p99 latency, unstable UX, and inability to maintain strict response SLOs.

4.4 Context Management Across Long Sessions

As conversation depth grows and multiple models are used, context consistency becomes harder. Potential failure modes include contradictory responses, stale state usage, and context window overflow.

4.5 Observability Fragmentation

With many models and services, observability can split across many logs and metrics. Without unified tracing and correlation, incident diagnosis becomes slow and governance evidence becomes weak.

4.6 Throughput & Concurrency Limits

Higher request concurrency stresses multiple service quotas simultaneously:

- Lambda concurrency limits
- CloudWatch Logs Insights query API quotas
- Amazon Bedrock throughput quotas
- API Gateway WebSocket management call limits

5 Recommended Multi-Model Architecture

5.1 Architecture Principles

- Keep user chat latency path lean
- Move heavy analytics and enrichment off the request path
- Centralise model routing policy
- Standardise event schema across all agents and models
- Prefer precomputed summaries for common questions

5.2 Proposed Logical Components

Component	Role	Key Behaviour
Ingestion Layer	Event normalisation	Collect runtime/evaluator/trace events continuously; normalise to common schema
Storage Layer (Hot + Cold)	Tiered data store	Hot store for low-latency operational lookups; cold store for historical analytics
Summary / Aggregation Layer	Precomputed views	Build per-user, per-session, per-model summaries asynchronously
Model Router (Control Plane)	Routing policy	Select model by intent, token budget, latency target, and cost policy
Analyzer Orchestrator	Context composition	Summary-first strategy; escalate to raw trace retrieval only when needed
Response Service	Delivery	Handle chunking, continuation tokens, and deterministic fallback responses

5.3 Suggested Data Stores

Store	Use Case
Amazon DynamoDB	Session and user-level hot lookup; summary objects with low-latency access
Amazon S3 + Athena	Historical event lake for ad hoc analytics and long-term retention
Amazon OpenSearch (Optional)	Fast full-text exploration across traces and prompts

5.4 Standard Event Schema (Target)

Every agent and model event should include the following standardised fields:

Field	Description
tenant_id	Multi-tenant isolation identifier (future use)
agent_id	Unique identifier for the agent source
model_id	Bedrock model ID used for the request
session_id	Chat session identifier
trace_id / request_id	End-to-end correlation identifiers
user_id / user_name	Authenticated user context
timestamp	ISO 8601 UTC event timestamp
intent_type	Classified intent from the planner
status / error_class	Request outcome and error categorisation
latency fields	Stage-level and end-to-end latency measurements
token usage fields	Input and output token counts
evaluator scores	Automated quality evaluation results
xray references	X-Ray trace and subsegment links

5.5 Model Routing Strategy

Intent Type	Model Profile	Rationale
summary / basic listing	Low-latency, lower-cost profile	Sufficient quality; minimises cost and latency
deep_dive / trace analysis	Higher reasoning model profile	Complex reasoning requires more capable model
timeout fallback	Short deterministic response or backup model	Guarantees a response even under model degradation

5.6 Reliability Pattern

PRIMARY ATTEMPT
Preferred model/profile — standard invocation with normal token budget.

SECONDARY ATTEMPT
Fallback model/profile invoked if primary latency breaches the configured threshold.

FINAL FALLBACK
Deterministic server-generated summary returned to user. Guarantees a response under any model degradation scenario.

6 Scalability Roadmap by Phase

Phase	Focus Area	Key Actions
Phase 1 — Stabilise	Immediate (current architecture)	Tighten routing/fallback; audit logs at key points only; add timeout ratio dashboard
Phase 2 — Summary-First	Short term	Introduce ingestion + hot summary tables; reduce direct CloudWatch dependency on hot path
Phase 3 — Multi-Model Router	Medium term	Externalise model selection policy; cost and latency-aware routing; automatic fallback chain
Phase 4 — Enterprise Scale	Long term	Multi-tenant isolation; cross-region resilience; governance and compliance audit package

7 Risks & Mitigations

Risk	Impact	Mitigation
Bedrock latency spikes	High — user-facing timeout	Lower max token budgets for time-sensitive intents; model fallback chain; deterministic fallback response
Query cost and performance degradation	High — cost and UX	Summary-first architecture; async aggregation; cold/hot storage split
Prompt drift across models	Medium — quality regression	Prompt templates versioned by intent and model family; eval-based regression tests per version
Data schema drift from multiple agent sources	Medium — data integrity	Strict event schema contract; ingestion validators and dead-letter handling
Operational blind spots	Medium — incident response	Correlation IDs end-to-end; unified metrics and tracing dashboard; SLO alerting

8 SLOs, KPIs & Frontend Reference

8.1 Target SLOs / KPIs

Category	Metric
Latency & Reliability	p50 response latency
Latency & Reliability	p95 response latency
Latency & Reliability	Timeout rate (%)
Latency & Reliability	Fallback response rate (%)
Quality & Usefulness	Answer success rate (no retries)
Quality & Usefulness	User follow-up correction rate
Quality & Usefulness	Trace deep-dive completeness score (internal rubric)
Cost & Efficiency	Cost per 1,000 requests
Cost & Efficiency	Average token usage by intent
Cost & Efficiency	Query cost by intent and timeframe

8.2 Frontend Key Source Files

Frontend stack: TypeScript + React 18 + Vite + Amplify Authenticator

File / Directory	Purpose
src/components/chat/ChatComposer	Message input bar with send button
src/components/chat/MessageList	Renders streamed assistant messages
src/components/chat/AnalyzerHelpPanel	Sample query suggestions panel
src/pages/AssistantChatPage	Assistant mode chat page
src/pages/AnalyzerChatPage	Analyzer mode chat page
src/hooks/useChat.ts	WebSocket connection + streaming state management
src/services/websocketClient	Raw WebSocket client with reconnect logic
src/services/auth	Cognito auth helpers via Amplify
src/config.ts	Environment variables + Amplify configuration
src/types/chat.ts	TypeScript interfaces for messages and WebSocket events

8.3 Authentication

Authentication is handled by AWS Cognito via the Amplify Authenticator component. Both username and email login are supported. The Cognito JWT token is attached to WebSocket connection handshake headers for backend authorisation.

8.4 Build Commands

Command	Purpose
cd frontend	Navigate to frontend directory
npm install	Install all dependencies
npm run dev	Development server at http://localhost:5173
npm run build	Production build output to frontend/dist/

9 Leadership Summary

EXECUTIVE SUMMARY

Sensei is functionally strong today for moderate load, with good intent-aware analysis and observability. The main scalability risk is request-time coupling to live query and model generation. The recommended path is a summary-first data architecture plus centralised model routing and deterministic fail-safes. This provides predictable latency, lower timeout risk, and controlled cost as models and data volume grow.

Key Recommendations

- **SHORT TERM: Keep current logging stages; keep deterministic fallback active; add explicit routing guard so trace-window requests do not fall into listing flow.**
- **MEDIUM TERM: Build ingestion and summary tables; introduce model-router module with policy configuration.**
- **LONG TERM: Formal multi-model control plane with A/B route policies; full scale test harness with synthetic high-volume telemetry.**

Related Documents

- PROJECT_DOCUMENTATION.md
- FUTURE_MULTIMODEL_ARCHITECTURE.md
- DEPLOYMENT.md